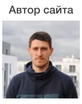
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50% на набор курсов:

Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 6. Builder, Factory

21 июня 2021

В этом уроке рассмотрим возможность кастомизации создания компонента с помощью билдера или фабрики.

Билдер

В прошлом уроке мы немного поговорили про билдер компонента. Билдер создается автоматически и содержит метод build и методы для передачи в компонент модулей, созданных вручную. Даггер дает нам возможность кастомизировать стандартный билдер.

Рассмотрим на простом примере:

```
@Component(modules = [AppModule::class, StorageModule::class, NetworkModule::class, MainModule
interface AppComponent {

    fun injectMainActivity(activity: MainActivity)

    @Component.Builder
    interface AppCompBuilder {
        fun buildAppComp(): AppComponent
    }

}
```

Прямо в интерфейсе компонента мы описываем интерфейс билдера - AppCompBuilder с аннотацией @Component.Builder. Теперь билдер, который даггер создаст для этого компонента, будет реализацией этого интерфейса.

В интерфейсе билдера мы, как минимум, должны описать build метод, который будет создавать компонент. Имя метода значения не имеет (например, buildAppComp), но он должен быть без аргументов и возвращать AppComponent. Даггер сгенерирует код этого метода так, чтобы он создавал компонент.

Используем свой билдер, чтобы создать компонент:

```
class App: Application() {

    lateinit var appComponent: AppComponent

    override fun onCreate() {
        super.onCreate()
        appComponent = DaggerAppComponent
            .builder()
            .buildAppComp()

    }
}
```

Метод builder() теперь возвращает наш кастомный билдер. И мы вызываем у него метод buildAppComp, чтобы получить компонент.

Модули

Чтобы кастомный билдер мог принимать от нас модули, созданные вручную, надо добавить соответствующие методы. Рассмотрим на примере модуля AppModule, который мы рассматривали на прошлом уроке:

```
@Module
class AppModule(private val context: Context) {

    @Provides
    fun getPreferences(): SharedPreferences {
        return context.getSharedPreferences("prefs", MODE_PRIVATE)
    }

    @Provides
    fun getResources(): Resources {
        return context.resources
    }

}
```

Описываем для этого модуля метод в интерфейсе нашего билдера:

```
@Component.Builder()
interface AppCompBuilder {
    fun buildAppComp(): AppComponent
    fun appModule(appModule: AppModule): AppCompBuilder
}
```

Метод должен принимать на вход объект AppModule, а на выходе возвращать билдер AppCompBuilder. Имя метода может быть любым, я использовал appModule.

Даггер создаст в билдере этот метод, и мы сможем его использовать для передачи модуля в компонент при создании компонента:

```
appComponent = DaggerAppComponent
    .builder()
    .appModule(AppModule(this))
    .buildAppComp()
```

Код вызывается в Application класса, поэтому this - это Context

BindsInstance

Ранее мы рассмотрели пример, когда с помощью конструктора AppModule мы передавали компоненту объект Context. Кастомный билдер дает нам возможность передачи объекта без участия модуля.

Для этого необходимо описать в кастомном билдере метод с аннотацией @BindsInstance:

```
@Component.Builder
interface AppCompBuilder {
    fun buildAppComp(): AppComponent

    @BindsInstance
    fun context(context: Context): AppCompBuilder
}
```

На вход он должен принимать объект, который мы хотим передать компоненту. А на выходе должен возвращать билдер.

Теперь Context можно убрать из конструктора AppModule. У компонента будет доступ к этому объекту напрямую, и он сможет передать его в Provides методы для получения SharedPreferences и Resources:

```
@Module
class AppModule {

    @Provides
    fun getPreferences(context: Context): SharedPreferences {
        return context.getSharedPreferences("prefs", MODE_PRIVATE)
    }

    @Provides
    fun getResources(context: Context): Resources {
        return context.resources
    }

}
```

Соответственно и создавать объект AppModule нам самим больше не надо. Билдер сам это сделает, конструктор теперь пустой.

Передаем Context напрямую в компонент с помощью кастомного билдера:

```
appComponent = DaggerAppComponent
    .builder()
    .context(this)
    .buildAppComp()
```

В документации указано, что этот способ является более предпочтительным, чем передача через конструктор модуля.

Factory

С версии 2.22 в даггере появился еще один способ создать компонент. Вместо билдера с несколькими методами, мы можем использовать один метод с несколькими аргументами. Для этого используется Factory.

Рассмотрим простой пример билдера:

```
@Component.Builder
interface AppCompBuilder {
    @BindsInstance fun context(context: Context): AppCompBuilder
    fun networkModule(networkModule: NetworkModule): AppCompBuilder
    fun buildAppComp(): AppComponent
}
```

У него есть метод для передачи объекта Context, метод для передачи NetworkModule и метод, который в итоге и создаст компонент.

Давайте вместо этого билдера напишем Factory с тем же функционалом:

```
@Component.Factory
interface AppCompFactory {
    fun create(@BindsInstance context: Context, networkModule: NetworkModule): AppComponent
}
```

Внутри компонента нам надо создать интерфейс с аннотацией @Component.Factory. Имя интерфейса может быть любым. А метод будет только один. Он должен принимать на вход все @BindsInstance объекты и модули, которые нужны компоненту. А на выходе он должен возвращать компонент. Т.е. получается билдер, упакованный в один метод.

Используем Factory для создания компонента:

```
appComponent = DaggerAppComponent
    .factory()
    .create(this, NetworkModule(...))
```

Для сравнения приведу код использования билдера:

```
appComponent = DaggerAppComponent
    .builder()
    .networkModule(NetworkModule(...))
    .context(this)
    .buildAppComp()
```

Выбирайте, какой способ вам нравится или подходит больше

Назад Вперёд

Комментарии

RE: Урок 6. Builder, Factory — **Николай** 31.12.2021 22:39
хорошо все объясни
Спасибо
Ответить

RE: Урок 6. Builder, Factory — **Николай** 31.12.2021 22:40
объяснил)
Ответить

RE: Урок 6. Builder, Factory — **Azat** 08.02.2022 15:34
.buildAppComp()
Он должен сразу этот метод находить? У меня почему-то никак не хочет
Ответить

RE: Урок 6. Builder, Factory — **Dmitry Vinogradov** 14.02.2022 14:59
Попробуйте пересобрать проект (CTRL + F9).
Если не помогает, то clean, invalidate cache и вот это все. Возможно, что-то где то закешировалось и не обновляется, такое бывает с даггером.
Ну и на всякий случай убедитесь, что интерфейс билдера помечен необходимой аннотацией.
Ответить

.buildAppComp() — **Александр** 11.02.2023 11:12
Тоже самое. Не показывает этот метод, даже после всех манипуляций по очистке кэша. В итоге прописал руками этот метод, красным он так и остался, но проект собрался. Метод все равно красный
Ответить

RE: Урок 6. Builder, Factory — **Лют** 17.03.2023 09:19
Пишут, что лучше использовать Фабрику, т.к. там нужно явно указывать все модули. В билдере, можно случайно забыть что-то указать.
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора
При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.